

OPDTrainer & DiffusionOPDTrainer: Complete Data Flow and Training Logic Analysis

Internal Technical Document

May 2026

Contents

1	Reference Configuration	2
2	OPDTrainer (trainers/opd/sde.py)	2
2.1	Initialization	2
2.2	Gradient Accumulation Steps (GAS)	2
2.3	Main Loop (start)	3
2.4	Phase 1: sample() — Trajectory Rollout	3
2.5	Phase 2: prepare_feedback() — Auxiliary Rewards	4
2.6	Phase 3: optimize(samples) — Training	4
2.6.1	Overall Structure	4
2.6.2	Teacher Aggregation Strategies	4
2.6.3	Pre-pass: _precompute_rank_reinforce_and_teacher	4
2.6.4	Main-pass: _optimize_train_pass (standard/average/round_robin)	5
2.6.5	Main-pass: _optimize_train_pass_sum (sum + route_by_source)	6
2.6.6	Main-pass: _optimize_train_pass_pcgrad	6
2.7	ODE-mode Loss Formula	6
2.8	Model Forward Count (per GPU per epoch)	7
2.9	Memory Profile	7
3	DiffusionOPDTrainer (trainers/diffusion_opd/trainer.py)	7
3.1	Initialization	7
3.2	Gradient Accumulation Steps (GAS)	8
3.3	Main Loop (start)	8
3.4	optimize() — Complete Training Logic	9
3.5	Balanced Sampling: _sample_balanced_batches	9
3.6	Teacher Weight Swap: _teacher_frozen_context	10
3.7	Loss Formula	10
3.8	Model Forward Count (per GPU per epoch)	10
3.9	Memory Profile	10
4	Comparative Analysis	11
4.1	Summary Table	11
4.2	GAS Computation Path	11
4.3	Trajectory Advancement	11

4.4	Data Isolation Semantics	12
4.5	On-Policy Semantics	12
5	Autocast Cache Handling	12
6	Boundary Conditions	13
6.1	Last Denoising Step ($j = 9$ / timestep_index=9)	13
6.2	latent_index_map (OPD only)	13
7	DeepSpeed Gradient Accumulation Interaction	13
7.1	Per-timestep accumulate() Requirement	13
7.2	PCGrad Exception	14
8	Conclusion	14

1 Reference Configuration

All analysis uses the following shared parameters:

Parameter	Symbol	Value
world_size	W	8
per_device_batch_size	B	8
unique_sample_num_per_epoch	S	576
group_size	G	1
num_inference_steps	N	10
num_teachers	M	3
gradient_step_per_epoch		1
num_inner_epochs		1
dynamics_type		ODE

Derived quantities:

$$\text{sample_num_per_iteration} = W \times B = 64$$

$$\text{num_batches_per_epoch} = \frac{S \times G}{W \times B} = \frac{576 \times 1}{64} = 9$$

2 OPDTrainer (trainers/opd/sde.py)

2.1 Initialization

```
self._is_ode = True # scheduler.dynamics_type == "ODE"
self.num_train_timesteps = num_inference_steps = 10
self._train_timestep_indices = [0, 1, ..., 9] # 10 items

# Load M=3 teacher LoRA snapshots into named-parameter slots
self._teacher_names = load_teachers(adapter, teacher_paths, device)
# Build teacher -> source routing map
self._teacher_sources[i] = set(teachers[i].sources) or None
```

2.2 Gradient Accumulation Steps (GAS)

$$\text{base_GAS} = \frac{\text{num_batches_per_epoch}}{\text{gradient_step_per_epoch}} = \frac{9}{1} = 9 \quad (1)$$

$$\text{get_num_train_timesteps}() = N = 10 \quad (\text{ODE mode}) \quad (2)$$

$$\text{GAS} = \text{base_GAS} \times N = 9 \times 10 = 90 \quad (3)$$

Meaning: every 90 calls to `accelerator.accumulate()` triggers one `optimizer.step()`.

2.3 Main Loop (start)

Algorithm 1 OPDTrainer.start()

```
while should_continue_training() do
    scheduler.set_seed(epoch + seed)
    // Optional: checkpoint, evaluation
    samples ← self.sample() {Phase 1: Rollout}
    self.prepare_feedback(samples) {Phase 2: Aux rewards (logging only)}
    self.optimize(samples) {Phase 3: Training}
    adapter.ema_step(epoch)
    epoch ← epoch + 1
end while
```

2.4 Phase 1: sample() — Trajectory Rollout

Algorithm 2 OPD sample() — ODE mode

```
adapter.rollout() {Switch to eval mode}
data_iter ← _interleaved_source_iter() {Round-robin across sources}
trajectory_indices ← compute_trajectory_indices([0..9], 10)
for  $b = 0, \dots, 8$  (9 batches) do
    batch ← next(data_iter) { $B = 8$  samples from one source}
    with torch.no_grad(), autocast():
        sample_batch ← adapter.inference(batch, compute_log_prob=True, ...)
        // Each sample stores: timesteps(10), all_latents(11), log_probs=None
        stitch_batch_metadata(batch, sample_batch) {Inject __source__}
        samples.extend(sample_batch)
end for
return samples {72 BaseSample objects}
```

Data iterator: `_interleaved_source_iter()` Round-robin over `sorted(train_dataloaders_by_source.keys)`

- batch 0: geneval → 8 samples
- batch 1: ocr → 8 samples
- batch 2: t2i_comp → 8 samples
- batch 3: geneval → 8 samples
- ... (repeating)

Each source contributes 24 samples/GPU/epoch.

The iterator injects `__source__` into both the batch dict and each sample’s `metadata` list for downstream teacher routing.

Output per sample:

- `timesteps`: shape (10,) — [1000, 900, ..., 100]
- `all_latents`: shape (11, C , $H/8$, $W/8$) — initial noise + 10 denoised states
- `latent_index_map`: shape ($T+1$,) — timestep index → `all_latents` index
- `log_probs`: None (ODE mode: `noise_level=0`, deterministic transition)
- `extra_kwargs["__source__"]`: dataset source tag

Memory: $72 \times 11 \times \text{latent_size} \approx 25$ GB (full trajectories stored; optionally offloaded to CPU).

2.5 Phase 2: `prepare_feedback()` — Auxiliary Rewards

OPD does *not* use external rewards for optimization. This phase:

1. Drains async reward workers (if configured), logs `aux_reward_*` via `nanmean/nanstd`
2. Logs first 30 sample images to wandb
3. Records teacher metadata for this epoch

2.6 Phase 3: `optimize(samples)` — Training

2.6.1 Overall Structure

Algorithm 3 OPD `optimize()` — high-level

```

adapter.train()
for inner_epoch = 0, ..., 0 (1 inner epoch) do
    shuffled_samples  $\leftarrow$  random_permute(72 samples)
    // Two-pass architecture:
    Pass 1 (no_grad): Compute all  $D_k$ ,  $\bar{R}_{k+1}$ , cache  $\mu_T$  per timestep
    Pass 2 (with_grad): Per-batch, per-timestep student forward + backward
end for
```

2.6.2 Teacher Aggregation Strategies

The `teacher_aggregation` parameter controls how multiple teachers are combined:

Strategy	Teachers/batch	Behavior
<code>round_robin</code>	1 (cycling)	$\text{teacher} = (\text{epoch} \times \text{inner} \times \text{batches} + \text{batch_idx}) \bmod M$
<code>average</code>	All M	Forward all teachers, average μ_T in latent space, single D_k
<code>sum</code>	All M	Per-teacher losses summed into single backward
<code>sum + route_by_source</code>	All M	Same as sum, but each teacher only computes loss on matching-source samples via boolean mask
<code>pcgrad</code>	All M	Per-teacher backward, PCGrad projection, manual grad accumulation

2.6.3 Pre-pass: `_precompute_rank_reinforce_and_teacher`

Iterates over all $9 \text{ micro-batches} \times 10 \text{ timesteps}$ under `no_grad`:

Algorithm 4 OPD Pre-pass (standard/average/round_robin)

```
for batch_idx = 0, ..., 8 do
  batch ← BaseSample.stack(shuffled_samples[start:end]) {8 samples}
  teacher_indices ← _teacher_indices_for_batch(batch_idx)
  for timestep_index ∈ [0, 1, ..., 9] do
    t ← batch.timesteps[:, timestep_index]
    t_next ← batch.timesteps[:, timestep_index+1] or 0
    x_k ← batch.all_latents[:, index_map[timestep_index]]
    μ_S ← student_forward(x_k, t, t_next).next_latents_mean
    μ_T ← teacher_forward(x_k, t, t_next).next_latents_mean {Averaged if multiple}
    D_k ← mean(‖μ_S − μ_T‖²) {Per-sample, shape (B,)}
    d_accum[k][start:end] ← D_k
  end for
end for
R̄_k ← reverse_cumulative(d_accum) {R̄_{k+1} = ∑_{j>k} D_j}
// Optional: group center/std normalization on R̄
```

2.6.4 Main-pass: _optimize_train_pass (standard/average/round_robin)

Algorithm 5 OPD Main-pass (standard)

```
for k = 0, ..., 9 (10 timesteps) do
  with accelerator.accumulate(*trainable_components):
    t, t_next, x_k, x_{k+1} ← from pre-stored trajectory
    student_out ← adapter.forward(x_k, t, t_next, compute_log_prob=True)
    μ_S ← student_out.next_latents_mean {WITH grad}
    μ_T ← mu_teacher_list[k] {Cached from pre-pass}
    D_k ← mean(‖μ_S − μ_T‖²)
    R̄_{k+1} ← r_per_k[k].detach()
    ℓ_path = D_k.mean()
    ℓ_rf = (R̄_{k+1} · log p_θ).mean()
    ℒ = α_path · ℓ_path + α_rf · ℓ_rf {+β_KL · KL if enabled}
    accelerator.backward(ℒ) {Activations freed}
    if sync_gradients: clip_grad → step → zero_grad
end for
```

Total `accumulate()` calls: 9 batches × 10 timesteps = 90 = GAS ✓

2.6.5 Main-pass: `_optimize_train_pass_sum` (sum + route_by_source)

Algorithm 6 OPD Main-pass (sum mode)

```

Precompute source masks:  $\text{mask}_m = (\text{sample.} \_\_\_\text{source} \_\_\_ \in \text{teacher}_m.\text{sources})$ 
for  $k = 0, \dots, 9$  do
  with accelerator.accumulate(*trainable_components):
     $\text{student\_out} \leftarrow \text{adapter.forward}(x_k, t, t_{\text{next}})$ 
     $\text{combined\_loss} \leftarrow 0$ 
    for  $m = 0, \dots, M - 1$  do
       $D_k^m \leftarrow \|\mu_S - \mu_T^m\|^2$  on  $\text{mask}_m$ -selected samples
       $\ell_m = \alpha_{\text{path}} \cdot D_k^m[\text{mask}_m].\text{mean}()$ 
      if  $\alpha_{\text{rf}} > 0$ :  $\ell_m += \alpha_{\text{rf}} \cdot (\bar{R}_{k+1}^m[\text{mask}_m] \cdot \log p[\text{mask}_m]).\text{mean}()$ 
       $\text{combined\_loss} += \ell_m$ 
    end for
     $\text{accelerator.backward}(\text{combined\_loss})$ 
end for

```

2.6.6 Main-pass: `_optimize_train_pass_pcgrad`

PCGrad uses a different GAS pattern: *one* `accumulate()` per batch (not per timestep). All T timesteps are accumulated internally via a manual gradient buffer.

Algorithm 7 OPD Main-pass (PCGrad mode)

```

with accelerator.accumulate(*trainable_components): {Once per batch}
   $\text{batch\_grad} \leftarrow [\text{zeros\_like}(p) \text{ for } p \text{ in params}]$ 
  for  $k = 0, \dots, 9$  do
     $\text{student\_out} \leftarrow \text{adapter.forward}(x_k)$ 
    Compute  $K$  per-teacher losses:  $\ell_0, \ell_1, \dots, \ell_{K-1}$ 
     $K$  backward passes (retain_graph)  $\rightarrow K$  grad snapshots
     $\text{projected} \leftarrow \text{pcgrad\_project}(\text{grad\_snapshots})$ 
     $\text{batch\_grad} += \text{projected}$ 
  end for
   $p.\text{grad} \leftarrow \text{batch\_grad} / T$ 
if sync_gradients: clip  $\rightarrow$  step  $\rightarrow$  zero_grad

```

PCGrad GAS = base_GAS = 9 (NOT multiplied by T ; internal T -step accumulation is manual).

2.7 ODE-mode Loss Formula

Under ODE dynamics: $\log p_\theta = 0$ (deterministic transition), so REINFORCE is inactive:

$$\mathcal{L} = \alpha_{\text{path}} \sum_{k=0}^{N-1} D_k(\theta) + \beta_{\text{KL}} \cdot \text{KL}(\theta \| \theta_{\text{ref}}) \quad (4)$$

where $D_k = \text{mean}(\|\mu_S(x_k, t_k) - \mu_T(x_k, t_k)\|^2)$.

With `route_by_source`:

$$\mathcal{L} = \alpha_{\text{path}} \sum_{k=0}^{N-1} \sum_{m=0}^{M-1} \text{mean}(D_k^m[\text{mask}_m]) \quad (5)$$

2.8 Model Forward Count (per GPU per epoch)

Phase	Student	Teacher	Mode
<code>sample()</code>	$72 \times 10 = 720$	0	<code>no_grad</code> (rollout)
Pre-pass	$72 \times 10 = 720$	$72 \times 10 \times K$	<code>no_grad</code> (train mode)
Main-pass	$72 \times 10 = 720$	0 (cached)	<code>with_grad</code>
Total	2160	720K	—

Table 1: K = active teachers per batch (depends on aggregation strategy).

2.9 Memory Profile

Phase	Peak Memory
After <code>sample()</code>	$72 \times 11 \times \text{latent_size} \approx 25$ GB (trajectories)
Pre-pass	+ 1 student forward (<code>no_grad</code> , no activations stored)
Main-pass peak	1 student forward activations + accumulated <code>.grad</code>
After backward	Activations freed, only <code>.grad</code> retained

3 DiffusionOPDTrainer (`trainers/diffusion_opd/trainer.py`)

3.1 Initialization

```
# Enforce ODE-only
assert adapter.scheduler.dynamics_type == "ODE"

# Load M=3 teacher LoRA snapshots
_teacher_names = load_teachers(adapter, [tc.path for tc in teachers], device)
_teacher_sources = [set(tc.sources) for tc in teachers]
num_tasks = 3

# Precompute timesteps (constant)
_timesteps = prepare_train_timesteps(scheduler, N=10, ...)
# -> tensor([1000, 900, 800, 700, 600, 500, 400, 300, 200, 100])
_num_steps = 10

# Per-source dataloader iterators for balanced sampling
_source_iters = {name: iter(dl) for ...}
```


3.2 Gradient Accumulation Steps (GAS)

$$\text{batches_per_task} = \lfloor \text{num_batches_per_epoch} / M \rfloor = \lfloor 9/3 \rfloor = 3 \quad (6)$$

$$\text{base_GAS} = \frac{\text{batches_per_task}}{\text{gradient_step_per_epoch}} = \frac{3}{1} = 3 \quad (7)$$

$$\text{get_num_train_timesteps}() = M \times N = 3 \times 10 = 30 \quad (8)$$

$$\text{GAS} = 3 \times 30 = 90 \quad (9)$$

Both trainers achieve $\text{GAS} = 90$, hence 1 `optimizer.step()` per epoch.

3.3 Main Loop (**start**)

Algorithm 8 DiffusionOPDTrainer.start()

```
while should_continue_training() do
    scheduler.set_seed(epoch + seed)
    // Optional: checkpoint, evaluation
    self.optimize() {Sampling + training combined}
    adapter.ema_step(epoch)
    epoch ← epoch + 1
end while
```

No separate `sample()` or `prepare_feedback()` phase.

3.4 optimize() — Complete Training Logic

Algorithm 9 DiffusionOPD optimize() — Algorithm 1

```

adapter.train()
torch.set_autocast_cache_enabled(False) {Teacher weight swap safety}
for round = 0, 1, 2 (batches_per_task rounds) do
  per_teacher_batches ← _sample_balanced_batches(device)
  {→ [geneval(8), ocr(8), t2i_comp(8)] — one batch per teacher}
  for m = 0, 1, 2 (M teachers) do
    batch ← per_teacher_batches[m]
    teacher_name ← _teacher_names[m]
    x ← sample_noise() {Initial Gaussian latent}
    for j = 0, ..., 9 (N = 10 denoising steps) do
      with accumulate(*trainable_components):
        with autocast():
          t ← timesteps[j]
          t_next ← timesteps[j + 1] if j + 1 < N else 0
          μS ← student_forward(x, t, t_next) {WITH grad}
          with no_grad(), teacher_frozen_context(teacher_name):
            μT ← teacher_forward(x, t, t_next) {Frozen}
            Dj =  $\frac{\alpha_{\text{path}}}{2} \cdot \text{mean}(\|\mu_S - \mu_T\|^2)$ 
            accelerator.backward(Dj) {Activations freed}
            x ← μS.detach() {Advance trajectory on-policy}
            if sync_gradients: clip → step → zero_grad
        end for
      end for
    end for
  end for

```

Total accumulate() calls: 3 rounds × 3 teachers × 10 steps = 90 = GAS ✓

3.5 Balanced Sampling: _sample_balanced_batches

```

def _sample_balanced_batches(self, device):
    per_teacher_batches = []
    for m in range(num_tasks): # 3
        src = list(_teacher_sources[m])[0] # First source for this teacher
        batch = _next_source_batch(src, device) # B=8 samples from that
            source
        per_teacher_batches.append(batch)
    return per_teacher_batches

```

Data allocation across 3 rounds × 3 teachers:

	Teacher 0 (geneval)	Teacher 1 (ocr)	Teacher 2 (t2i_comp)
Round 0	geneval(8)	ocr(8)	t2i_comp(8)
Round 1	geneval(8)	ocr(8)	t2i_comp(8)
Round 2	geneval(8)	ocr(8)	t2i_comp(8)
Total	24	24	24

Each teacher sees **exactly** 24 samples/GPU/epoch from its assigned source. No cross-source contamination.

3.6 Teacher Weight Swap: `_teacher_frozen_context`

```
@contextmanager
def _teacher_frozen_context(self, name):
    info = adapter._named_parameters[name]
    live = adapter._get_component_parameters(info.target_components)
    saved_flags = [p.requires_grad for p in live]
    try:
        with adapter.use_named_parameters(name): # .data.copy_() swap
            for p in live:
                p.requires_grad_(False)
            yield
    finally:
        for p, flag in zip(live, saved_flags):
            p.requires_grad_(flag)
```

Critical: `use_named_parameters` swaps via `.data.copy_()` which preserves `data_ptr`. The autocast cache (keyed by `data_ptr`) must be disabled to prevent serving stale cached weights from a previous teacher.

3.7 Loss Formula

$$\mathcal{L} = \frac{\alpha_{\text{path}}}{2} \sum_{\text{round}} \sum_{m=0}^{M-1} \sum_{j=0}^{N-1} \text{mean}(\|\mu_S(x_j) - \mu_T^m(x_j)\|^2) \quad (10)$$

Each teacher computes loss *only* on its own source batch (natural isolation, no mask needed). No REINFORCE term. No KL anchor.

3.8 Model Forward Count (per GPU per epoch)

Component	Forward Count	Mode	Total
Student	$3 \times 3 \times 10 = 90$	with_grad	90
Teacher	$3 \times 3 \times 10 = 90$	no_grad	90
Total		180	

3.9 Memory Profile

Moment	Memory
Per-timestep peak	1 student fwd (activations) + 1 teacher fwd (no activations)
After backward	Activations freed; only <code>.grad</code> + current x (1 latent)
Entire epoch	$O(1)$ latent storage (only current x)

4 Comparative Analysis

4.1 Summary Table

	OPDTrainer	DiffusionOPDTrainer
Sampling	Separate <code>sample()</code> phase	Inline in <code>optimize()</code>
Trajectory storage	Full (25 GB)	$O(1)$ (1 latent)
GAS	$9 \times 10 = 90$	$3 \times 30 = 90$
<code>accumulate()</code> mode	Per-timestep	Per-timestep
<code>optimizer.step()</code> /epoch	1	1
Student fwd/epoch	2160	90
Teacher fwd/epoch	720K	90
Peak memory	25 GB + 1 fwd	1 student fwd + 1 teacher fwd
Supports SDE	✓	× (ODE only)
Supports REINFORCE	✓	× (pathwise only)
Supports KL anchor	✓	×
Teacher aggregation	5 strategies	Fixed per-teacher isolation
Source routing	Probabilistic (shuffle)	Deterministic (guaranteed)
Samples/teacher/GPU	~24 (probabilistic)	24 (exact)
Total samples/GPU/epoch	72	72

4.2 GAS Computation Path

Both trainers follow the same computation chain:

```
# In Arguments.__post_init__():
# 1. TrainingArguments.__post_init__(world_size):
num_batches_per_epoch = (576 * 1) / (8 * 8) = 9
gradient_accumulation_steps = compute_gradient_accumulation_steps(9)
# -> OPD: 9 / 1 = 9
# -> DiffusionOPD: batches_per_task / 1 = 3 / 1 = 3

# 2. Arguments._adjust_gradient_accumulation():
num_train_timesteps = get_num_train_timesteps(args)
# -> OPD: 10 (N for ODE)
# -> DiffusionOPD: M * N = 3 * 10 = 30
gradient_accumulation_steps *= num_train_timesteps
# -> OPD: 9 * 10 = 90
# -> DiffusionOPD: 3 * 30 = 90
```

4.3 Trajectory Advancement

OPD: Pre-stored trajectory (off-policy latents).

- `sample()` generates full trajectory with the student at epoch start (eval mode)
- `optimize()` reads latents from `all_latents[:, index_map[k]]`
- Main-pass student forward uses the *same* fixed latents as pre-pass
- The latent x_k does NOT change even as the student's predictions differ

DiffusionOPD: Online trajectory (on-policy latents).

- No separate rollout; trajectory generated inline

- After each step: $x \leftarrow \mu_S.\text{detach}()$
- Each step’s input is the *current* student’s output from the previous step
- Gradient does not backpropagate across steps (detach boundary)

Semantic difference: Since no `optimizer.step()` occurs within an epoch (both trainers step exactly once at the end), the student weights are identical throughout. Under `eval≈train` (no dropout), both approaches produce the same latent trajectory within a single epoch. The difference only manifests when `num_inner_epochs > 1`: OPD reuses the epoch-start trajectory while DiffusionOPD would regenerate (though current config uses `inner_epochs=1`).

4.4 Data Isolation Semantics

OPD (`sum + route_by_source=true`):

1. `sample()` collects 72 samples in round-robin source order
2. `optimize()` **shuffles** all 72 samples randomly
3. Each micro-batch may contain mixed-source samples
4. Per-teacher mask filters: only matching-source samples contribute to that teacher’s loss
5. Some batches may have 0 samples for a given teacher (handled via dummy backward)

DiffusionOPD:

1. Each round explicitly samples one batch per teacher from its source dataloader
2. Teacher m ’s batch contains *only* source- m samples (by construction)
3. No masking needed; natural isolation
4. Every teacher always sees exactly B effective samples per round

4.5 On-Policy Semantics

	OPD	DiffusionOPD
Trajectory generator	Student (eval mode, <code>sample()</code>)	Student (train mode, <code>optimize()</code>)
Trajectory consumer	Student (train mode, main-pass)	Same forward (inline)
Policy consistency	Weak off-policy ¹	Fully on-policy
Gradient scope	Per-timestep (detached)	Per-timestep (detached)

5 Autocast Cache Handling

Both trainers must disable the autocast weight cache when performing teacher LoRA swaps:

OPD: Disables in pre-pass (`_precompute_d_per_timestep_pcgrad`) around the teacher swap loop. Main-pass does not re-run teachers (uses cached μ_T), so no issue.

¹“Weak off-policy”: the student weights don’t change between `sample()` and `optimize()` within the same epoch, so the policy is identical. Truly off-policy only when `num_inner_epochs > 1`.

DiffusionOPD: Disables for the *entire* `optimize()` scope, since every timestep performs a teacher swap.

```
# DiffusionOPD pattern:
prev_cache = torch.is_autocast_cache_enabled()
torch.set_autocast_cache_enabled(False)
try:
    # Full training loop (3 rounds x 3 teachers x 10 steps)
finally:
    torch.set_autocast_cache_enabled(prev_cache)
```

Why: `use_named_parameters` swaps via `.data.copy_()` preserving `data_ptr`. The autocast cache keys on `data_ptr`, so it would serve `teacher_0`'s cached cast to `teacher_1`'s forward.

6 Boundary Conditions

6.1 Last Denoising Step ($j = 9$ / `timestep_index=9`)

Both trainers handle the terminal step identically:

```
t = timesteps[9] # = 100 (last non-zero timestep)
t_next = torch.tensor(0, device=device) # Terminal: clean image

# adapter.forward(t=100, t_next=0, latents=x)
# -> Predicts final denoised result mu
```

This matches the inference loop in `SD3_5Adapter.inference()`:

```
for i, t in enumerate(timesteps): # i=0..9
    t_next = timesteps[i+1] if i+1 < len(timesteps) else tensor(0)
    output = self.forward(t=t, t_next=t_next, latents=latents, ...)
```

6.2 `latent_index_map` (OPD only)

`all_latents` has 11 states (indices 0–10):

- Index 0: initial noise x_0
- Index 1–10: denoised states after each of the 10 steps

`latent_index_map[timestep_index]` maps training timestep index to the corresponding `all_latents` position. For the last step:

- `latents = all_latents[:, map[9]]` — input to step 9
- `next_latents = all_latents[:, map[10]]` — output of step 9 (final result)

7 DeepSpeed Gradient Accumulation Interaction

7.1 Per-timestep `accumulate()` Requirement

DeepSpeed's `engine.backward()` internally triggers `engine.step()` when the gradient accumulation counter reaches the configured GAS value. Each `accumulate()` context increments this counter by 1.

Correct pattern (both trainers use this):

```

for timestep in timesteps:
    with accelerator.accumulate(*components): # Counter += 1
        loss = compute_loss(timestep)
        accelerator.backward(loss) # Single backward per accumulate

```

Wrong pattern (would cause premature steps):

```

with accelerator.accumulate(*components): # Counter += 1 (once)
    for timestep in timesteps:
        loss = compute_loss(timestep)
        accelerator.backward(loss) # T backwards, but only 1 counter
                                   increment!

```

7.2 PCGrad Exception

PCGrad mode handles T timesteps manually within a single `accumulate()` and sets `p.grad` directly (bypassing `accelerator.backward`). Its `GAS = base_GAS = 9` (not multiplied by T).

8 Conclusion

Both trainers guarantee:

- **1 optimizer step per epoch** with the reference configuration
- **10 model forwards per sample** matching the inference loop
- **Identical effective batch size:** 576 samples across 8 GPUs
- **Per-timestep backward:** $O(1)$ activation memory

The key architectural trade-offs:

1. **Memory vs. flexibility:** OPD stores full trajectories (25 GB) enabling REINFORCE and multi-pass optimization; DiffusionOPD achieves $O(1)$ storage via interleaved rollout.
2. **Compute:** OPD requires $12\times$ more student forwards (2160 vs. 180 total) due to the separate sample/pre-pass/main-pass architecture.
3. **Data isolation:** DiffusionOPD guarantees per-teacher sample isolation by construction; OPD requires `route_by_source` masking (probabilistic per-batch counts after shuffle).
4. **Generality:** OPD supports SDE dynamics, REINFORCE, KL anchoring, and 5 teacher aggregation strategies. DiffusionOPD is ODE-only, pathwise-only, with fixed per-teacher isolation.